

RELATIONAL DATABASE

A **relational database** is a way of storing data in **tables** made up of rows and columns. Each table holds related information, and different tables can be **connected to each other** through relationships. These relationships help show how data in one table is linked to data in another.

Relational databases make it easy to **organize, manage, and retrieve data** in a consistent and secure way. Because of this structured and reliable approach, they are **widely used by businesses and organizations**.

Relation:

A table in a database is called a **relation**. For example, **Student** is the name of a relation.

Attributes:

The **columns** of a table are called attributes. They describe the properties of an entity.

Example: Register Number, Name, City, Age.

Tuples:

Each **row** in a table is called a tuple. If a table has 4 rows, it has 4 tuples.

Degree:

The number of attributes (columns) in a table is called the **degree**.

Example: If the Student table has 4 columns, its degree is 4.

Cardinality:

The number of tuples (rows) in a table is called **cardinality**.

Example: If the Student table has 4 rows, its cardinality is 4.

Primary Key:

A **primary key** is a unique identifier for each record in a table.

Foreign Key:

A **foreign key** is used to connect one table with another by referring to the primary key of another table.

Schema in Relational Databases

A **schema** is like a blueprint of a database. It shows how tables, fields, relationships, and rules are organized.

Importance of Schema:

- **Data Integrity:** Keeps data accurate and reliable.
- **Relationships:** Defines how tables are connected.
- **Normalization:** Organizes data to reduce duplication and improve accuracy.

Schemas also allow setting rules like **unique values, not-null values, and default values**, which help keep data consistent and improve querying.

Key Features of Relational Databases

- **Flexibility:** Easy to add or modify tables and fields.
- **Scalability:** Can handle large amounts of data efficiently.
- **ACID Properties:** Ensures reliable transactions:
 - **Atomicity:** All or nothing
 - **Consistency:** Data remains valid
 - **Isolation:** Transactions don't interfere
 - **Durability:** Data is saved permanently

Advantages of Relational Databases

- Organizes data in a structured way, ensuring **accuracy and consistency**.
- Uses **SQL**, which makes data searching and retrieval easy.
- Supports **reliable transactions** through ACID properties.

Disadvantages of Relational Databases

- May need **more storage space** due to repeated data.
- Not very efficient for **unstructured data** like images, videos, or audio files.
- Managing relationships becomes **more complex** as the database size increases.

Purpose of Database System

- It is used to **store manage, and retrieve data efficiently**
- It ensures **data accuracy, security, and consistency**
- Allows **easy querying using SQL**,
- Manages **relationships between data**
- Supports **multiple users at the same time**
- Provides **backup and recovery** in case of failures
- Database systems can **scale with growing data** and help in **data analysis and reporting**.
- They are essential for effective data management in all types of organizations.

Database System Applications

Relational databases are widely used in many areas:

- **Business:** CRM and ERP systems
- **E-commerce:** Product management and order processing
- **Finance:** Banking and investment systems
- **Healthcare:** Electronic health records and pharmacy systems
- **Telecommunications:** Billing and network management
- **Education:** Student information and learning management systems
- **Social Media:** User profiles and content management
- **Government:** Public records and census data
- **Data Warehousing:** Business intelligence and reporting
- **Web Applications:** Content management systems and API services

The concepts of data abstraction, instances, and schemas

1. Data Abstraction

Data abstraction means hiding complex details of how data is stored and managed, and showing only what the user needs to see.

It helps make databases easier to use, secure, and maintain.

There are **three levels of data abstraction**:

a) Physical Level (Lowest Level)

- Describes **how data is actually stored** in the database.
- Deals with storage details like:
 - Files
 - Indexes

- Data blocks
- Storage devices
- This level is mostly handled by **database administrators**, not end users.

Example:

How student records are stored on disk, file organization, and indexing methods.

b) Logical Level (Middle Level)

- Describes **what data is stored** and **how data items are related**.
- Focuses on:
 - Tables
 - Attributes (columns)
 - Relationships
 - Constraints (primary key, foreign key, etc.)
- Does **not** care about physical storage.

Example:

A STUDENT table with fields like Roll_No, Name, Course, and Marks.

c) View Level (Highest Level)

- Shows **only required data** to users.
- Different users can have **different views** of the same database.
- Helps in **data security** and **simplicity**.

Example:

- A student can view only their marks.
- An admin can view all student details.

2. Instances

A **database instance** refers to the **actual data present in the database at a specific moment in time**.

- It keeps **changing** as data is:
 - Inserted
 - Updated
 - Deleted
- It represents the **current state** of the database.

Example:

Today, the STUDENT table contains records of 500 students.
Tomorrow, after adding 10 students, the instance changes.

🔑 **Schema is fixed, instance changes frequently.**

3. Schemas

A **schema** is the **overall design or structure** of the database.

- It defines:

- Tables
- Columns
- Data types
- Relationships
- Constraints
- It acts as a **blueprint** of the database.
- Schemas **rarely change**.

Types of Schemas:

a) Database Schema

- Defines the **entire database structure**.
- Includes all tables, relationships, and constraints.

Example:

Definition of STUDENT, COURSE, and EXAM tables and how they are linked.

b) External Schema (View Schema)

- Defines how **different users see the data**.
- Customised according to user needs.

Example:

- Teacher view: student names and marks.
- Accountant view: student fees details only.

Data Models in Database Management System (DBMS)

A **data model** in a DBMS defines **how data is structured, stored, related, and accessed** in a database.

It acts as a **blueprint** that guides database design and helps users understand how data is organized.

Different data models are used based on application needs.

1. Hierarchical Data Model

Structure:

- Data is organized in a **tree-like structure**.
- Each child has **only one parent**, but a parent can have many children.
- Follows **one-to-many** relationships.

Usage:

- Suitable for systems with a clear hierarchy.

Example:

- Organization structure
Company → Departments → Employees

Limitation:

- Difficult to represent many-to-many relationships.

2. Network Data Model

Structure:

- Similar to hierarchical model, but allows **multiple parent nodes**.
- Supports **many-to-many relationships**.
- Uses graph-like structure.

Usage:

- Used where data relationships are more complex.

Example:

- Students enrolled in multiple courses.

Advantage:

- More flexible than hierarchical model.

3. Relational Data Model

Structure:

- Data is stored in **tables (relations)**.
- Tables consist of **rows (records)** and **columns (attributes)**.
- Relationships are created using **keys**.

Usage:

- Most widely used data model in DBMS.
- Uses **SQL (Structured Query Language)**.

Key Features:

- Data integrity
- Normalization
- Easy querying

Example:

- STUDENT(Roll_No, Name, Course)
- COURSE(Course_ID, Course_Name)

4. Object-Oriented Data Model

Structure:

- Data is stored as **objects**, similar to object-oriented programming.
- Objects contain:
 - Data (attributes)
 - Methods (functions)

Usage:

- Useful for complex applications like multimedia, CAD, and engineering systems.

Example:

- An object Student with attributes and functions like calculateGrade().

5. Entity-Relationship (ER) Model

Structure:

- Uses **entities**, **attributes**, and **relationships**.
- Represented using **ER diagrams**.

Usage:

- Mainly used during the **database design phase**.
- Helps convert requirements into a database schema.

Example:

- Entity: Student
- Relationship: Enrolls
- Entity: Course

6. Document Data Model

Structure:

- Data is stored as **documents** in formats like **JSON** or **XML**.
- Schema is **flexible**.

Usage:

- Common in **NoSQL databases** such as MongoDB.
- Suitable for semi-structured data.

Example:

- A user profile stored as a JSON document.

7. Key-Value Data Model

Structure:

- Data is stored as **key-value pairs**.
- Very simple structure.

Usage:

- Used in caching systems and fast data retrieval.

Example:

- Key: UserID_101
- Value: {Name: "Anita", Age: 22}

Advantage:

- High performance and fast access.

8. Column-Family Data Model

Structure:

- Data is stored in **columns instead of rows**.
- Columns are grouped into **column families**.

Usage:

- Used in distributed and big-data systems.
- Common in databases like **Apache Cassandra**.

Advantage:

- Efficient for handling large datasets.

9. Graph Data Model**Structure:**

- Data is represented as:
 - **Nodes** (entities)
 - **Edges** (relationships)
- Relationships are first-class elements.

Usage:

- Ideal for applications with complex relationships.

Example:

- Social networks (friends, followers)
- Recommendation systems

Database Architecture

Database Architecture refers to the way a database system is designed and how different components such as the user, application, and database interact with each other.

Depending on usage requirements, DBMS architecture is mainly classified into:

1. **1-Tier Architecture**
2. **2-Tier Architecture**
3. **3-Tier Architecture**

1-Tier Architecture

In **1-Tier Architecture**, the **user, application, and database all reside on the same machine**. The user directly interacts with the database system without any network connection.

This architecture is simple and mainly used for **personal or standalone applications**.

Example:**Microsoft Excel Spreadsheet**

- User interface, application logic, and data storage are all handled on one system.
- The user enters data, performs calculations, and stores the data locally.

Advantages of 1-Tier Architecture:

- **Simple Architecture:** Easy to understand and maintain.
- **Cost-Effective:** No need for extra hardware or servers.
- **Easy to Implement:** Suitable for small projects and individual use.

2-Tier Architecture

2-Tier Architecture follows a **client–server model**.

Here, the client directly communicates with the database server using interfaces like **ODBC or JDBC**.

- **Client Side:** User interface and application logic.

- **Server Side:** Database storage, query processing, and transaction management.

Example:

Library Management System (School or Small Organization)

- **Client Layer (Tier 1):**
 - Used by librarians or users.
 - Allows searching books, issuing books, and checking due dates.
- **Database Layer (Tier 2):**
 - Stores book details, user data, and transaction records.

The client sends requests to the database server, which processes them and returns results.

Advantages of 2-Tier Architecture:

- **Easy Access:** Faster data retrieval.
- **Scalable:** Can add more clients or upgrade server hardware.
- **Low Cost:** Cheaper than 3-Tier architecture.
- **Easy Deployment:** Simple to install and manage.
- **Simple Structure:** Only two layers make it easy to understand.

3-Tier Architecture

In **3-Tier Architecture**, an **additional middle layer (application server)** is introduced between the client and the database server.

- The client **does not directly communicate** with the database.
- The application server handles:
 - Business logic
 - Query processing
 - Transaction management

This architecture is commonly used in **large web-based applications**.

Example:

E-Commerce Website

- **Presentation Layer (Client):**
 - User searches products and places orders.
- **Application Layer (Middle Tier):**
 - Checks product availability
 - Calculates price and discounts
- **Database Layer:**
 - Stores product details, user data, and order history

Advantages of 3-Tier Architecture:

- **Enhanced Scalability:** Load can be distributed across application servers.
- **Improved Security:** Clients cannot directly access the database.
- **Data Integrity:** Middle layer helps prevent data corruption.

Disadvantages of 3-Tier Architecture:

- **More Complex:** Compared to 2-Tier architecture.
- **Difficult Interaction:** Communication becomes slower due to extra layers.

Comparison Table (Very Useful for Exams)

Architecture Layers		Usage	Complexity
1-Tier	User + DB	Personal applications	Very Low
2-Tier	Client + Server	Small organizations	Medium
3-Tier	Client + App + DB	Large web applications	High

Database Users and Administrators

A database system is accessed and managed by different types of users. Each user has a specific role to ensure efficient and secure use of data.

Database Users

Types of Database Users

1. End Users

- These are **ordinary users** who interact with the database through applications or user interfaces.
- They do not write complex queries.
- They mainly perform tasks like viewing, inserting, or updating data.

Example:

Students checking marks, customers using banking or shopping apps.

2. Power Users

- Users with **advanced database knowledge**.
- They can write complex SQL queries, generate reports, and perform data analysis.
- Often involved in decision-making processes.

Example:

Data analysts, managers, or system analysts.

Permissions

- Database users are assigned **specific permissions** based on their roles.
- Permissions control:
 - Data access
 - Data modification
 - Execution of queries
- This ensures **security and data integrity**.

Database Interfaces

Database interfaces provide ways for users and applications to interact with the database.

1. Graphical User Interfaces (GUIs)

- Provide a **visual and user-friendly** way to work with databases.
- Allow users to create tables, run queries, and manage data easily.

Examples:

SQL Server Management Studio (SSMS), MySQL Workbench

2. Command-Line Interfaces (CLIs)

- Allow users to interact with the database by writing **SQL commands directly**.
- Preferred by advanced users and administrators.

Examples:

MySQL command line, PostgreSQL terminal

3. APIs (Application Programming Interfaces)

- Enable **external applications** to interact with databases programmatically.
- Used in web and mobile applications.

Example:

A website connecting to a database using Java, Python, or PHP APIs.

Database Administrators (DBAs)

A **Database Administrator (DBA)** is responsible for the overall management and maintenance of the database system.

Roles and Responsibilities of DBAs

- **Installation and Configuration:**
Installing and setting up the DBMS software.
- **Performance Monitoring:**
Monitoring database performance and optimizing queries.
- **Backup and Recovery:**
Ensuring data is backed up regularly and restored during system failures.
- **Security Management:**
Managing user access, permissions, and protecting data from unauthorized access.
- **Troubleshooting:**
Identifying and resolving database errors and failures.

Introduction to the Relational Model

The **Relational Model** represents data using a collection of **tables**, also known as **relations**. Each table consists of **rows and columns**.

- **Columns (Attributes):** Define the type of data stored.
- **Rows (Tuples):** Represent actual records or real-world entities.

The relational model is a **record-based data model**, where data is stored in fixed-format records with a predefined structure.

Characteristics of the Relational Model

1. Data Representation

- Data is organized in **tables (relations)**.

- Rows are called **tuples** and columns are called **attributes**.

2. Atomic Values

- Each cell in a table contains **only one value**.
- No multivalued or nested data is allowed.

3. Unique Keys

- Every table has a **primary key**.
- It uniquely identifies each record and avoids duplication.

4. Attribute Domain

- Each attribute has a **defined domain**.
- The domain specifies:
 - Data type
 - Range of values
 - Constraints

5. Tuples as Rows

- Each row (tuple) represents one **real-world entity** or relationship.

6. Relation Schema

- Defines the **structure of a table**.
- Includes:
 - Table name
 - Attributes
 - Data types and constraints

7. Data Independence

- Supports:
 - **Logical data independence**
 - **Physical data independence**
- Changes in schema do not affect application programs.

8. Integrity Constraints

The relational model enforces rules to maintain accuracy and consistency:

- **Domain Constraints:**
Attribute values must belong to the defined domain.
- **Entity Integrity:**
Primary key values cannot be NULL.
- **Referential Integrity:**
Foreign key values must match a primary key in another table or be NULL.

9. Relational Operations

Supports powerful operations such as:

- Selection
- Projection
- Join
- Union
- Intersection

These operations help retrieve and manipulate data efficiently.

10. Data Consistency

- Constraints reduce:
 - Data redundancy
 - Update anomalies
- Ensures accurate and reliable data.

11. Set-Based Representation

- Tables are treated as **sets**.
- Operations are based on **mathematical set theory**.

Database Schema

A **database schema** is the logical design or structure of a database. It defines **how data is organized** and **how different data elements are related** to each other.

It acts as a **blueprint** that specifies tables, fields, relationships, and rules used to manage data.

The schema is created by **database designers** to ensure that data is **consistent, efficient, and easy to maintain**.

Key Points of Database Schema

- Defines the **logical organization of data**, including tables, fields, and relationships.
- Describes relationships between entities using **primary keys and foreign keys**.
- Converts unstructured data into a **well-organized structured format**.
- Guides how data is **stored, accessed, modified, and maintained**.
- Helps users and administrators clearly understand the database structure.

Types of Database Schemas

1. Physical Database Schema

- Defines **how data is physically stored** in the storage system.
- Includes:
 - File organization
 - Indexes
 - Storage blocks

- Specifies the **actual code and syntax** used to create database structures.
- Decided and maintained by the **Database Administrator (DBA)**.
- Represents the **lowest level of abstraction**.

Example:

How tables are stored on disk, indexing methods, and storage locations.

2. Logical Database Schema

- Defines the **logical structure** of data.
- Includes:
 - Tables
 - Attributes
 - Relationships
 - Views
 - Integrity constraints
- Designed using **Entity–Relationship (ER) modeling**.
- Ensures data is organized properly and maintains **data integrity**.
- Does **not deal with physical storage details**.
- Represents a **higher level of abstraction** than physical schema.

Keys in Database

Keys are used to **uniquely identify records** and establish relationships between tables.

Primary Key

Definition:

A unique identifier for each record in a table.

Characteristics:

- Must be **unique**
- Cannot contain **NULL**
- Only **one primary key per table**

Foreign Key

Definition:

A field in one table that refers to the **primary key of another table**.

Characteristics:

- Establishes relationships between tables
- Can contain **NULL values**
- Maintains **referential integrity**

Unique Key

Definition:

Ensures that all values in a column are unique.

Characteristics:

- Can allow **NULL values**
- A table can have **multiple unique keys**

Composite Key

Definition:

A primary key made up of **two or more columns**.

Use:

- When a single attribute is not enough to uniquely identify a record.

Candidate Key

Definition:

A set of attributes that can uniquely identify records.

Characteristics:

- Must be unique
- One candidate key is chosen as the **primary key**

Surrogate Key

Definition:

An **artificially created key**, usually a sequence number.

Characteristics:

- Not derived from real-world data
- Ensures uniqueness easily

Natural Key

Definition:

A key derived from **real-world data**.

Example:

Aadhar Number, Passport Number

Relational Query Language (RQL)

Relational Query Language is used to **retrieve and manipulate data** in relational databases. The most commonly used RQL is **SQL (Structured Query Language)**.

- It allows users to communicate with the database.
- Based on **relational algebra and relational calculus**.
- Can be **procedural** or **non-procedural**.

Types of Relational Query Language

1. **Procedural Query Language**
2. **Non-Procedural Query Language**

Procedural Query Language

- User specifies **what data to retrieve and how to retrieve it**.
- Executes operations **step by step**.
- Gives instructions to DBMS in sequence.

Relational Algebra

Relational Algebra is a **procedural query language** that uses operations on relations.

Relational Algebra Operators

- **Select (σ)**: Filters rows
Example: $\sigma_{age>30}(\text{Employees})$
- **Projection (π)**: Selects specific columns
Example: $\pi_{name, age}(\text{Employees})$
- **Set Difference ($-$)**: Rows in one relation but not in another
Example: $\text{Employees} - \text{Managers}$
- **Cartesian Product ($\times$)**: Combines all rows of two relations
Example: $\text{Employees} \times \text{Departments}$
- **Union ($\cup$)**: Combines tuples from both relations
Example: $\text{Employees} \cup \text{Managers}$
- **Intersection ($\cap$)**: Common tuples
Example: $\text{Employees} \cap \text{Managers}$

Join Operations in Relational Algebra

Natural Join ($\bowtie$)

- Joins relations using common attributes
- Removes duplicate columns

Theta Join ($\bowtie_{\theta}$)

- Join based on any comparison condition

Equi-Join

- A type of theta join using equality condition

Outer Joins

- **Left Outer Join ($\bowtie_L$)**: All left tuples
- **Right Outer Join ($\bowtie_R$)**: All right tuples
- **Full Outer Join ($\bowtie_{full}$)**: All tuples from both relations

Non-Procedural Query Language

- User specifies **what data is required**, not how to retrieve it.
- Only one query is written.
- Easier to use and understand.

Relational Calculus

Relational Calculus is a **non-procedural query language**.

Types:

- **Tuple Relational Calculus (TRC)**:
Selects tuples that satisfy a condition.

- **Domain Relational Calculus (DRC):**
Selects values based on attribute domains.

Procedural vs Non-Procedural Language

Procedural Language	Non-Procedural Language
Command-driven	Function-driven
Specifies steps	Specifies result
Complex semantics	Simple semantics
Larger programs	Smaller programs
High efficiency	Lower efficiency
Uses loops & recursion	Uses recursion
Not suitable for time-critical apps	Suitable for time-critical apps

Entity–Relationship (ER) Model

The **Entity–Relationship (ER) model** is a **conceptual data model** used to design databases in a **Relational Database Management System (RDBMS)**. It visually represents **real-world entities**, their **attributes**, and the **relationships** among them. The ER model is mainly used during the **database design phase**, before converting the design into relational tables.

Key Components of the ER Model in RDBMS

1. Entities and Entity Sets

Entities

- An **entity** is a real-world object or concept that can be uniquely identified.
- Each entity represents a **single instance**.

Examples: Employee, Student, Department

Entity Sets

- An **entity set** is a collection of similar entities.
- In RDBMS, entity sets are represented as **tables**.

Example:

An Employee table where each row represents one employee.

2. Attributes

Attributes describe the **properties or characteristics** of an entity.

Types of Attributes

- **Simple Attributes:**
Cannot be divided further.
Example: FirstName, Age
- **Composite Attributes:**
Can be broken into smaller parts.
Example: FullName → FirstName + LastName

- **Derived Attributes:**
Calculated from other attributes.
Example: Age derived from DateOfBirth

3. Relationships and Relationship Sets

Relationships

- A **relationship** defines how entities are connected to each other.

Example:

An employee *works for* a department.

Relationship Sets

- A collection of similar relationships.
- In RDBMS, relationships are implemented using **foreign keys**.

Example:

A DepartmentID foreign key in the Employee table represents the *Works_For* relationship.

Example ER Model

Entities:

- **Employee:** EmployeeID (PK), FirstName, LastName, DepartmentID (FK)
- **Department:** DepartmentID (PK), DepartmentName

Relationship:

- One department can have **many employees**
- Each employee belongs to **one department**
→ **One-to-Many (1:N) relationship**

Constraints in ER Model

a) Mapping Cardinalities

Mapping cardinality defines **how many entities** are related to each other.

Types:

- **One-to-One (1:1):**
One entity relates to one entity.
- **One-to-Many (1:N):**
One entity relates to many entities.
- **Many-to-One (N:1):**
Many entities relate to one entity.
- **Many-to-Many (M:N):**
Many entities relate to many entities.

b) Key Constraints

- Specifies attributes that **uniquely identify an entity**.
- Represented using **primary keys**.

Example:

EmployeeID uniquely identifies an employee.

c) Participation Constraints

Defines whether participation of an entity in a relationship is **mandatory or optional**.

Types:

- **Total Participation:**
Every entity must participate (shown using double lines).
- **Partial Participation:**
Some entities may not participate.

E–R Diagrams

An **ER Diagram** is a graphical representation of entities, attributes, and relationships.

1. Basic Structure

- **Entities:** Rectangles
- **Attributes:** Ovals
- **Relationships:** Diamonds

2. Complex Attributes

- Attributes that can be divided into sub-parts.
- Represented by ovals connected to smaller ovals.

Example:

Address → Street, City, State, ZipCode

3. Roles

- Used when the **same entity participates in a relationship in different roles**.

Example:

In a *Manages* relationship, an employee can be a **manager** or a **subordinate**.

4. Non-Binary Relationship Sets

- Relationships involving **more than two entities**.

Example:

Student – Course – Instructor (ternary relationship)

5. Weak Entity Set

Definition:

A **weak entity** cannot be uniquely identified on its own and depends on a **strong entity**.

Characteristics:

- Does **not have a primary key**
- Uses a **partial key (discriminator)**
- Depends on a strong entity for existence

Representation:

- Weak entity → **Double rectangle**
- Identifying relationship → **Double diamond**
- Partial key → **Dotted underline**

Example:

- Strong Entity: Loan
- Weak Entity: Payment
- Partial Key: PaymentNumber
- Relationship: LoanPayment

ER to Relational Mapping

This process converts an **ER diagram into relational tables** that can be implemented in an RDBMS.

Step-by-Step Mapping Process

Step 1: Map Entities to Tables

- Each entity becomes a **table**.
- Attributes become **columns**.
- Primary key becomes the **table's primary key**.

Step 2: Map Multivalued Attributes

- Create a **separate table**.
- Include:
 - Primary key of the original entity
 - Multivalued attribute
- Combined attributes form the **primary key**.

Step 3: Map Relationships

- **One-to-One:**
Add primary key of one entity as foreign key in the other.
- **One-to-Many:**
Add primary key of the “one” side as foreign key in the “many” side.
- **Many-to-Many:**
Create a **new table** with:
 - Foreign keys from both entities
 - Composite primary key

Step 4: Map Weak Entities

- Create a table for the weak entity.
- Add primary key of the strong entity as foreign key.
- Combine it with the partial key to form the **primary key**.

Step 5: Define Integrity Constraints

- **Primary Key Constraints**
- **Foreign Key Constraints**
- **Check Constraints** (range, conditions)

ER to Relational Mapping Example

ER Diagram:

- **Student:** StudentID (PK), Name, Age
- **Course:** CourseID (PK), CourseName
- **Enrollment:** Many-to-Many, Attribute: Grade

Relational Tables:

Student Table

StudentID (PK), Name, Age

Course Table

CourseID (PK), CourseName

Enrollment Table

StudentID (FK), CourseID (FK), Grade

→ Composite Primary Key: (StudentID, CourseID)

Extended ER (EER) Features in RDBMS

The **Extended Entity–Relationship (EER) model** enhances the basic ER model by introducing advanced concepts to represent complex real-world situations more accurately. The major EER features include **Specialization**, **Generalization**, **Attribute Inheritance**, **Constraints on Generalization**, and **Aggregation**.

1. Specialization

Specialization is the process of dividing a higher-level entity (superclass) into one or more lower-level entities (subclasses) based on distinguishing characteristics.

Key Points:

- It is a **top-down approach**.
- Subclasses have **specific attributes or relationships** in addition to those inherited from the superclass.
- It improves clarity and data organization.

Example:

Consider a **Person** entity.

- **Person** (PersonID, Name, DateOfBirth)
- **Student** (StudentID, Degree)
- **Employee** (EmployeeID, Department)

Here, *Student* and *Employee* are specializations of *Person*.

2. Generalization

Generalization is the reverse of specialization. It combines multiple similar lower-level entities into a single higher-level entity.

Key Points:

- It is a **bottom-up approach**.
- Common attributes are moved to the superclass.
- Reduces redundancy in database design.

Example:

- **Student** (StudentID, Major)
- **Teacher** (TeacherID, Subject)

Generalized into:

- **Person** (PersonID, Name, DateOfBirth)

Here, *Student* and *Teacher* become subclasses of *Person*.

3. Attribute Inheritance

Attribute inheritance allows subclasses to automatically inherit all attributes and relationships of their superclass.

Key Points:

- Avoids duplication of attributes.
- Ensures consistency across entities.

Example:

If **Person** has attributes (*Name*, *DateOfBirth*), then:

- **Student** inherits (*Name*, *DateOfBirth*) and adds (*Degree*)
- **Employee** inherits (*Name*, *DateOfBirth*) and adds (*Department*)

4. Constraints on Generalization and Specialization

Constraints define **rules** that control how entities participate in specialization or generalization.

a) Disjoint vs Overlapping

- **Disjoint Constraint:**
An entity can belong to **only one subclass**.
Example: A person can be either a **Student** or an **Employee**, but not both.
- **Overlapping Constraint:**
An entity can belong to **multiple subclasses**.
Example: A person can be both a **Student** and an **Employee**.

b) Total vs Partial Participation

- **Total Specialization:**
Every superclass entity **must** belong to at least one subclass.
Example: Every person must be either a Student or an Employee.
- **Partial Specialization:**
A superclass entity **may or may not** belong to a subclass.
Example: A person may be neither a Student nor an Employee.

5. Aggregation

Aggregation is a higher-level abstraction that allows a **relationship set to be treated as an entity**.

Key Points:

- Used when a relationship itself needs to participate in another relationship.

- Helps represent complex associations clearly.

Example:

Entities:

- **Employee** (EmployeeID, Name)
- **Project** (ProjectID, ProjectName)
- **Department** (DepartmentID, DepartmentName)

Relationship:

- **WorksOn** (Employee works on Project with Hours)

Using aggregation, **WorksOn** is treated as an entity to relate it with **Department**, forming:

- **ProjectAssignment** (WorkID, ProjectID, DepartmentID)

Conclusion

The **EER model** extends the basic ER model by supporting advanced concepts such as specialization, generalization, inheritance, constraints, and aggregation. These features help in designing **flexible, accurate, and real-world-oriented database systems**, especially for complex applications.